



॥ नवोन्मेषेण देशनिर्माणम् ॥
ఐఐటీ హైదరాబాద్
आई आई टी हैदराबाद
IIT Hyderabad

INDIAN INSTITUTE OF TECHNOLOGY HYDERABAD

Department of Electrical Engineering - ICDT

PROJECT REPORT

Arduino UNO-Based Digital Clock -V1

Submitted by:

Ganesh Balotiya (EE26BTECH11205)

Akshith Badavath (EE26BTECH11204)

Submitted To:

Dr. GVV Sharma

Teaching Assistants:

Vivek Kumar, Subhodeep Chakraborty, Jnanesh

Date of Submission:

August 28, 2026

Contents

Declaration and Acknowledgement	4
Abstract	5
1 Introduction: What We Set Out to Make	6
1.1 Our Project in One Page	6
1.2 Objectives	7
1.3 Scope and an Important Safety Note	8
2 What the Finished Project Can Do	9
3 Components and Their Jobs	11
3.1 Why the Display, Driver, and Resistors Matter	11
4 Design and Working Principle	14
4.1 Functional Flow	14
4.2 Clock Data	14
4.3 BCD Representation	15
4.4 Six-Digit Display Arrangement	15
4.5 Display Control and Multiplexing	16
4.6 Project Flowchart	18
4.7 How the Modes are Organised	19
5 Assembly and Further Testing	20
5.1 Construction Procedure	20
5.2 A Note About Careful Changes	25
6 Program Logic	26
6.1 Core Program Responsibilities	26
6.2 Arduino Pin Configuration	27

6.3	Detailed Electrical Connections	28
6.4	Timekeeping Logic	28
6.5	Button Edge Detection	29
6.6	Edit Modes	29
6.7	Increment and Decrement Logic	30
6.8	Pause and Resume	31
6.9	Digit Separation	31
6.10	BCD Generation	32
6.11	Display Refresh	32
7	Testing, Observations, and Troubleshooting	34
7.1	How We Tested	34
7.2	Useful Troubleshooting Ideas	35
8	Results	39
9	What We Learned as a Team	40
9.1	Technical Learning	40
9.2	Working Together	40
10	Our Project Development Diary	42
10.1	Small Steps Made the Final Result Easier	42
10.2	How We Kept Each Other Moving	42
11	Conclusion	43
11.1	Final Project Snapshot	43
11.2	Present Limitations	44
11.3	Future Improvements	44
	References	46
A	Final Circuit Diagram	46

B Final Hardware Photographs	47
C Complete Arduino Source Code	49

Declaration and Acknowledgement

Declaration

We hereby declare that this project report titled “*Arduino UNO-Based Digital Clock - V1*” presents the work carried out by us as part of our academic project at the Indian Institute of Technology Hyderabad.

The project was developed through practical experimentation, hardware assembly, circuit testing, debugging, and software implementation. The observations and results presented in this report are based on our project development and testing process.

Team Members:

- Ganesh Balotiya – EE26BTECH11205
- Akshith Badavath – EE26BTECH11204

Acknowledgement

We would like to express our sincere gratitude to **Dr. GVV Sharma** for providing us with the opportunity to work on this project and for his guidance throughout the development process.

We also thank the teaching assistants, **Vivek Kumar, Subhodeep Chakraborty, and Jnanesh**, for their technical guidance and support during the practical sessions.

We are grateful to the institute for providing the laboratory equipment and components required to construct and test the project.

Finally, we would like to thank everyone who contributed directly or indirectly to the successful completion of this project.

Abstract

This project presents the design and implementation of an Arduino UNO-based digital clock using six seven-segment displays. The clock represents time in the **hours:minutes:seconds** (**HH:MM:SS**) format.

The project combines a microcontroller, BCD-to-seven-segment decoding, LED display interfacing, push-button inputs, and display multiplexing. The Arduino UNO acts as the central controller, while the 7447 decoder is used to convert Binary Coded Decimal (BCD) inputs into the corresponding seven-segment display signals.

The clock provides four user-control functions. A Select button is used to choose between normal operation and editing of hours, minutes, or seconds. Increment and decrement buttons modify the selected value, while a fourth button provides pause and resume functionality.

A major part of the project involved practical hardware debugging. Several issues were encountered during construction, including incorrect segment behaviour, display flickering, wiring errors, display-selection problems, and incorrect representation of some digits, particularly 6 and 9. These problems were resolved through individual component testing, checking pin connections, verifying logic levels, and systematically isolating faulty parts of the circuit.

The final system demonstrates the practical application of digital logic, microcontroller programming, seven-segment display control, multiplexing, and hardware troubleshooting in the construction of a functional digital clock.

1 Introduction: What We Set Out to Make

Digital clocks are a simple but useful example of the interaction between digital electronics and embedded systems. Although a digital clock appears to be a simple device, its implementation involves several important concepts such as timing, numerical representation, display interfacing, decoding, input handling, and circuit debugging.

The objective of this project was to construct a working digital clock using an Arduino UNO and six seven-segment displays. The six display positions represent the hours, minutes, and seconds.

The project was developed as a practical hardware project rather than only a software simulation. Therefore, considerable attention was given to wiring, component orientation, current limiting, display behaviour, and troubleshooting.

1.1 Our Project in One Page

The basic operation of the project can be summarised as follows:

1. The Arduino UNO acts as the main controller.
2. The clock keeps track of hours, minutes, and seconds.
3. The numerical values are represented as decimal digits.
4. Each decimal digit is converted into BCD.
5. The 7447 decoder converts the BCD information into seven-segment control signals.
6. The corresponding seven-segment display shows the digit.
7. Multiplexing is used to control the six display positions.
8. Push buttons provide user interaction for selection, increment, decrement, and pause/resume operations.

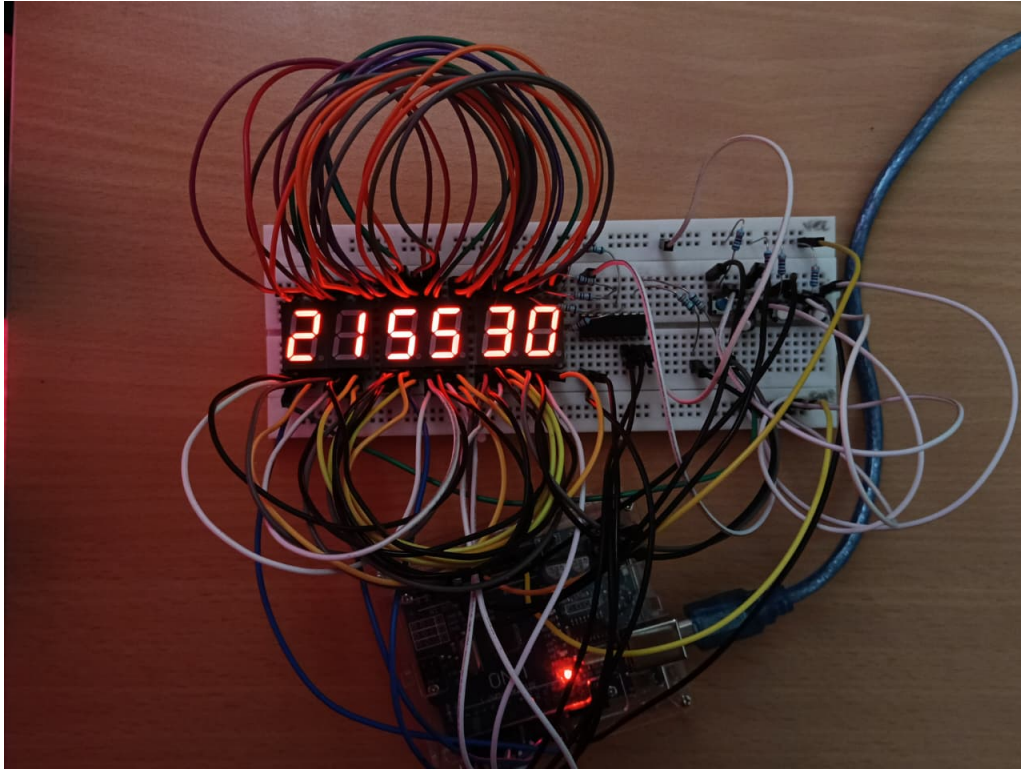


Figure 1: Final working digital clock

1.2 Objectives

The major objectives of the project were:

- To understand the working principle of a seven-segment display.
- To interface seven-segment displays with an Arduino UNO.
- To understand Binary Coded Decimal representation.
- To understand the operation of a 7447 BCD-to-seven-segment decoder/driver.
- To construct a six-digit digital clock.
- To implement hours, minutes, and seconds display.
- To interface push buttons with the microcontroller.
- To understand display multiplexing.
- To develop practical hardware debugging skills.
- To test the completed circuit systematically.

1.3 Scope and an Important Safety Note

The project focuses on the educational implementation of a digital clock using discrete hardware components and a microcontroller.

During construction, power should be disconnected before making major wiring changes. LED segments should be operated with appropriate current-limiting resistors, and the circuit should be checked carefully before applying power.

The project is intended as a low-voltage educational electronics system and should be operated using the specified laboratory components and supply conditions.

2 What the Finished Project Can Do

The completed project provides a digital representation of time using six seven-segment display positions.

The display arrangement is:

HH : MM : SS

where:

- HH represents hours,
- MM represents minutes,
- SS represents seconds.

The project demonstrates the complete path from numerical time data inside the micro-controller to a visible six-digit output.

Main Features

- Six-digit time display.
- HH:MM:SS time format.
- Arduino UNO-based control.
- 7447 BCD decoder interface.
- Six-digit multiplexed display control.
- Four push-button controls.
- Hour, minute, and second adjustment.
- Pause/resume functionality.
- Blinking indication during edit mode.
- Single increment/decrement action per button press.

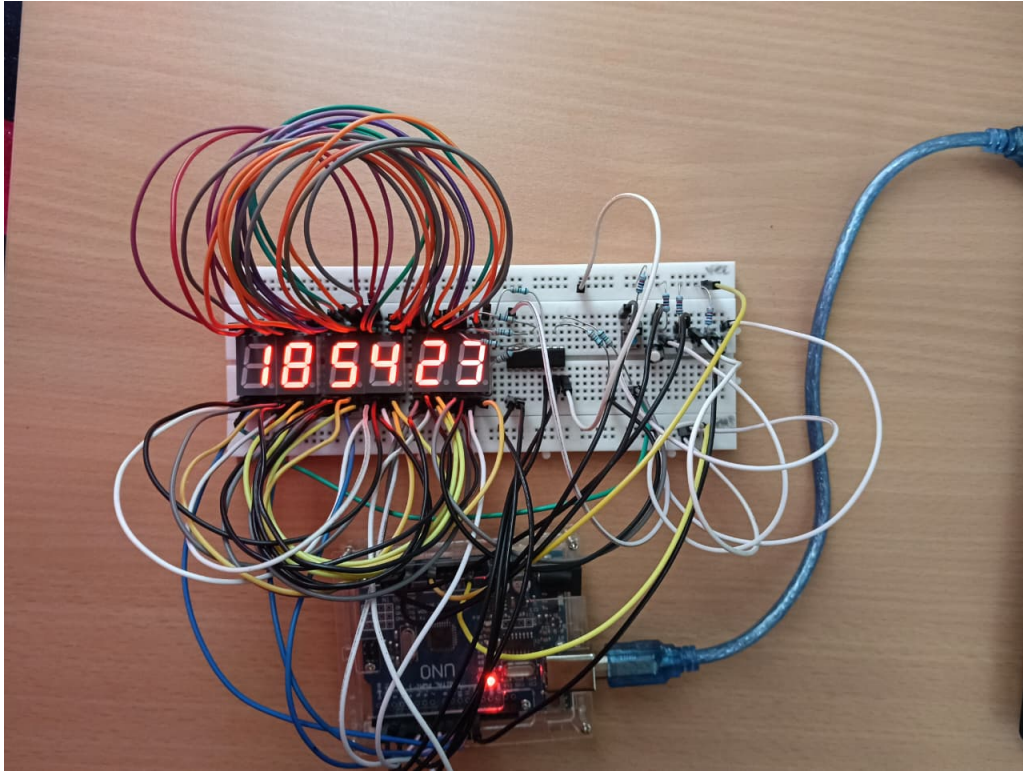


Figure 2: Final working digital clock

3 Components and Their Jobs

The following components were used during the development of the project.

Table 1: Major components used in the project

Component	Purpose
Arduino UNO	Main microcontroller and control unit
Seven-segment displays	Visual representation of numerical digits
7447 IC	BCD-to-seven-segment decoder/driver
Resistors	Limit current through LED segments
Push buttons	Provide user input and clock control
Breadboard	Temporary circuit construction platform
Jumper wires	Electrical connections between components
USB/power supply	Provides power and programming connection

3.1 Why the Display, Driver, and Resistors Matter

Seven-Segment Display

A seven-segment display consists of seven individually controllable LED segments conventionally labelled:

$$a, b, c, d, e, f, g$$

By activating appropriate combinations of these segments, decimal digits from 0 to 9 can be displayed.

For example, the digit 8 requires all seven segments to be active, whereas the digit 1 requires only the appropriate right-side segments.



Figure 3: display

7447 Decoder

The 7447 is a BCD-to-seven-segment decoder/driver. It accepts a four-bit BCD input and generates the corresponding outputs required to represent the decimal digit on a compatible seven-segment display.

The four BCD input bits represent the binary weights:

$$8, \quad 4, \quad 2, \quad 1$$

For example:

$$5 = 0101$$

and

$$9 = 1001$$

The use of a decoder simplifies the process of generating the seven-segment pattern for each decimal digit.

The laboratory material also demonstrates the use of the 7447 for seven-segment display

control and Arduino-based BCD interfacing.

Current-Limiting Resistors

Seven-segment displays contain LEDs. LEDs require current limiting to prevent excessive current from flowing through the segments.

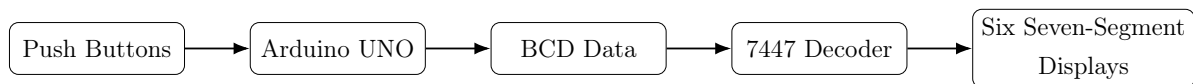
Resistors are therefore included in the segment paths according to the circuit design.

4 Design and Working Principle

The digital clock consists of several functional blocks working together.

4.1 Functional Flow

The overall functional flow can be represented as:



...

Figure 4: Functional block diagram of the digital clock

4.2 Clock Data

The clock maintains three primary quantities:

$$H = \text{hours}$$

$$M = \text{minutes}$$

$$S = \text{seconds}$$

The displayed time can therefore be expressed as:

$$T = HH : MM : SS$$

The seconds value increases once every second during normal operation. When seconds reach 59, they return to zero and the minutes are incremented.

Similarly, when minutes reach 59, they return to zero and the hour value is incremented.

The hour counter operates from 00 through 23.

Therefore:

$$59 \text{ seconds} \rightarrow 00 \text{ seconds}, \quad M \rightarrow M + 1$$

and:

$$59 \text{ minutes} \rightarrow 00 \text{ minutes}, \quad H \rightarrow H + 1$$

After:

$$23 : 59 : 59$$

the clock returns to:

$$00 : 00 : 00$$

4.3 BCD Representation

Each decimal digit is represented by four binary bits.

Table 2: BCD representation of decimal digits

Decimal Digit	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

4.4 Six-Digit Display Arrangement

The six display positions are logically arranged as:

$$\mathbf{H_1} \quad \mathbf{H_0} \quad \mathbf{M_1} \quad \mathbf{M_0} \quad \mathbf{S_1} \quad \mathbf{S_0}$$

These represent:

HH : MM : SS

The six individual digits are obtained from the corresponding hour, minute, and second values.

For example, if:

$$H = 12, \quad M = 34, \quad S = 56$$

then:

$$D = [1, 2, 3, 4, 5, 6]$$

and the display shows:

$$\boxed{12 : 34 : 56}$$

4.5 Display Control and Multiplexing

A major practical aspect of the project is controlling six display positions.

The displays are controlled sequentially. At a particular instant, one display position is enabled and its corresponding BCD data is provided. The selected display is then disabled before moving to the next display.

The conceptual sequence is:

$$D_1 \rightarrow D_2 \rightarrow D_3 \rightarrow D_4 \rightarrow D_5 \rightarrow D_6 \rightarrow D_1 \rightarrow \dots$$

In the implemented program, each display is enabled for a short period of approximately 2 milliseconds. The rapid repetition of this process makes the six displays appear continuously illuminated to the observer. For the implemented program, each of the six display positions is activated for approximately 2 ms. Therefore, one complete refresh cycle takes approximately

$$T_{\text{refresh}} = 6 \times 2 \text{ ms} = 12 \text{ ms}.$$

The approximate refresh frequency of the complete six-digit display is therefore

$$f_{\text{refresh}} = \frac{1}{T_{\text{refresh}}} = \frac{1}{0.012} \approx 83.3 \text{ Hz}.$$

This sufficiently rapid sequential switching makes the individual display activations appear as a continuously illuminated display to the observer.

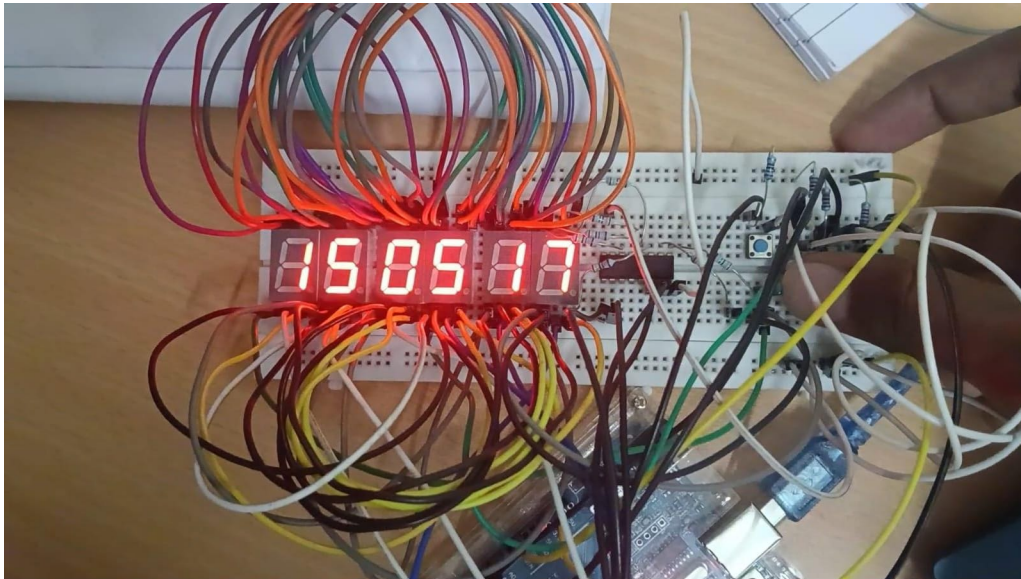
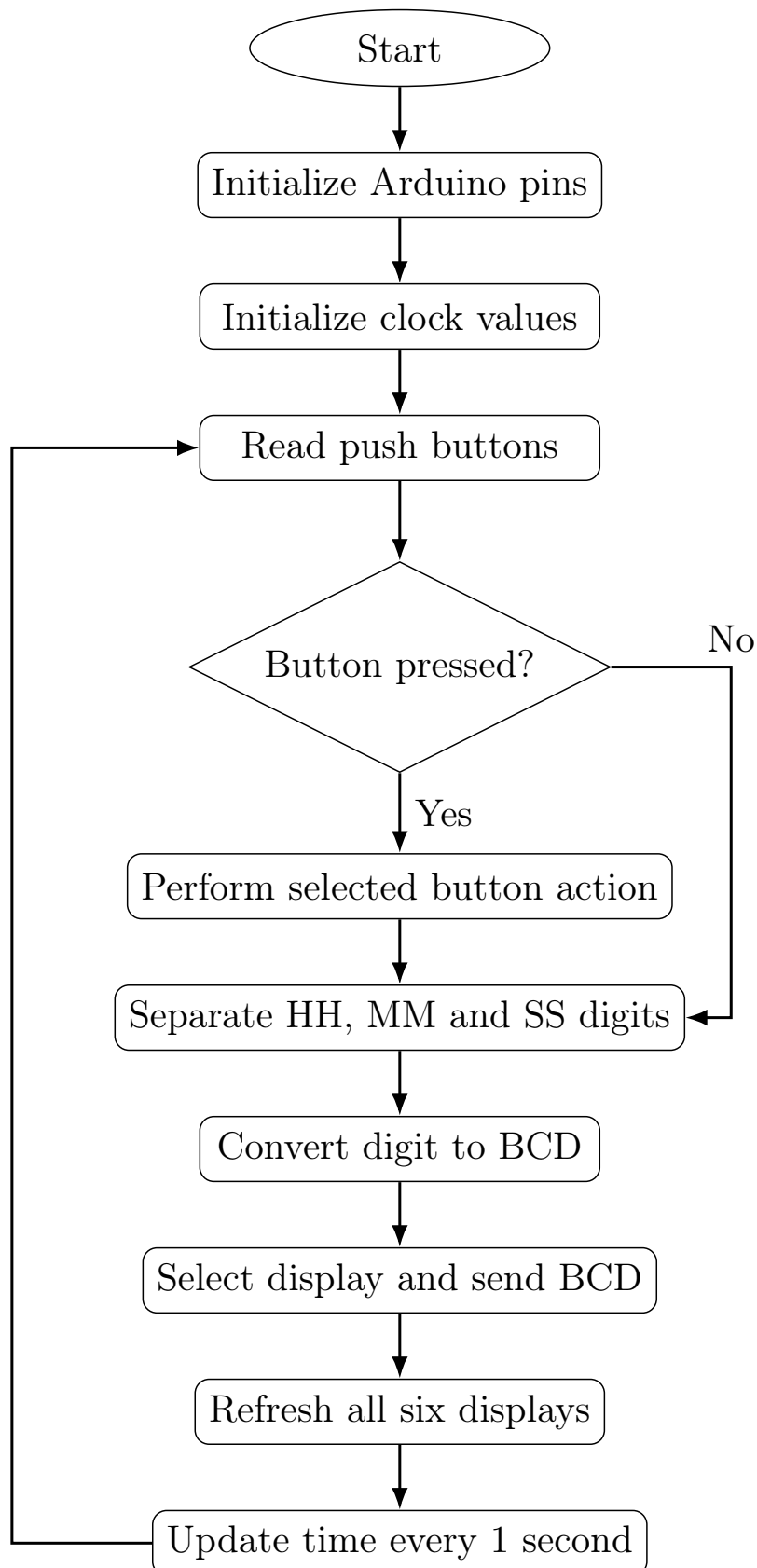


Figure 5: Display multiplexing

4.6 Project Flowchart

The complete control sequence is shown below.



4.7 How the Modes are Organised

The clock software uses four operating states.

1. **Normal Run Mode:** The clock continuously increments the time.
2. **Edit Hours Mode:** The hour digits can be increased or decreased.
3. **Edit Minutes Mode:** The minute digits can be increased or decreased.
4. **Edit Seconds Mode:** The second digits can be increased or decreased.

The Select button cycles through these four states:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$$

The currently selected pair of digits blinks during editing.

5 Assembly and Further Testing

5.1 Construction Procedure

The hardware was developed progressively instead of connecting all components at once.

Step 1: Component Identification

The first step was to identify the Arduino UNO, seven-segment displays, 7447 IC, resistors, push buttons, breadboard, and jumper wires.

Particular attention was given to identifying the correct display configuration and corresponding common connection.

Step 2: Testing a Single Display

Before attempting the complete clock, one seven-segment display was tested independently.

The purpose was to confirm:

- Correct segment identification.
- Correct common connection.
- Correct logic behaviour.
- Correct digit formation.

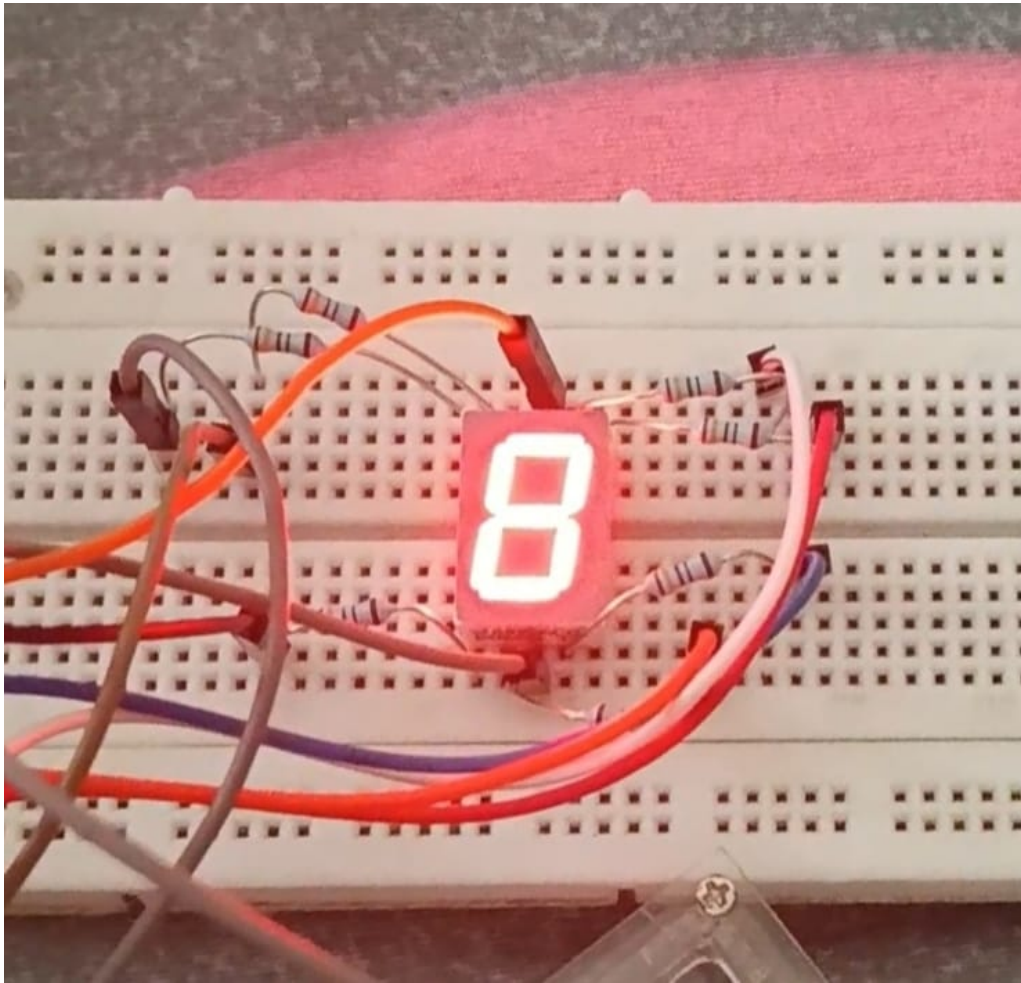


Figure 6: Initial single seven segment display test

Step 3: Testing Digits

The individual digits from 0 to 9 were tested.

This stage was important because it exposed incorrect segment mappings before the complete six-digit system was assembled.

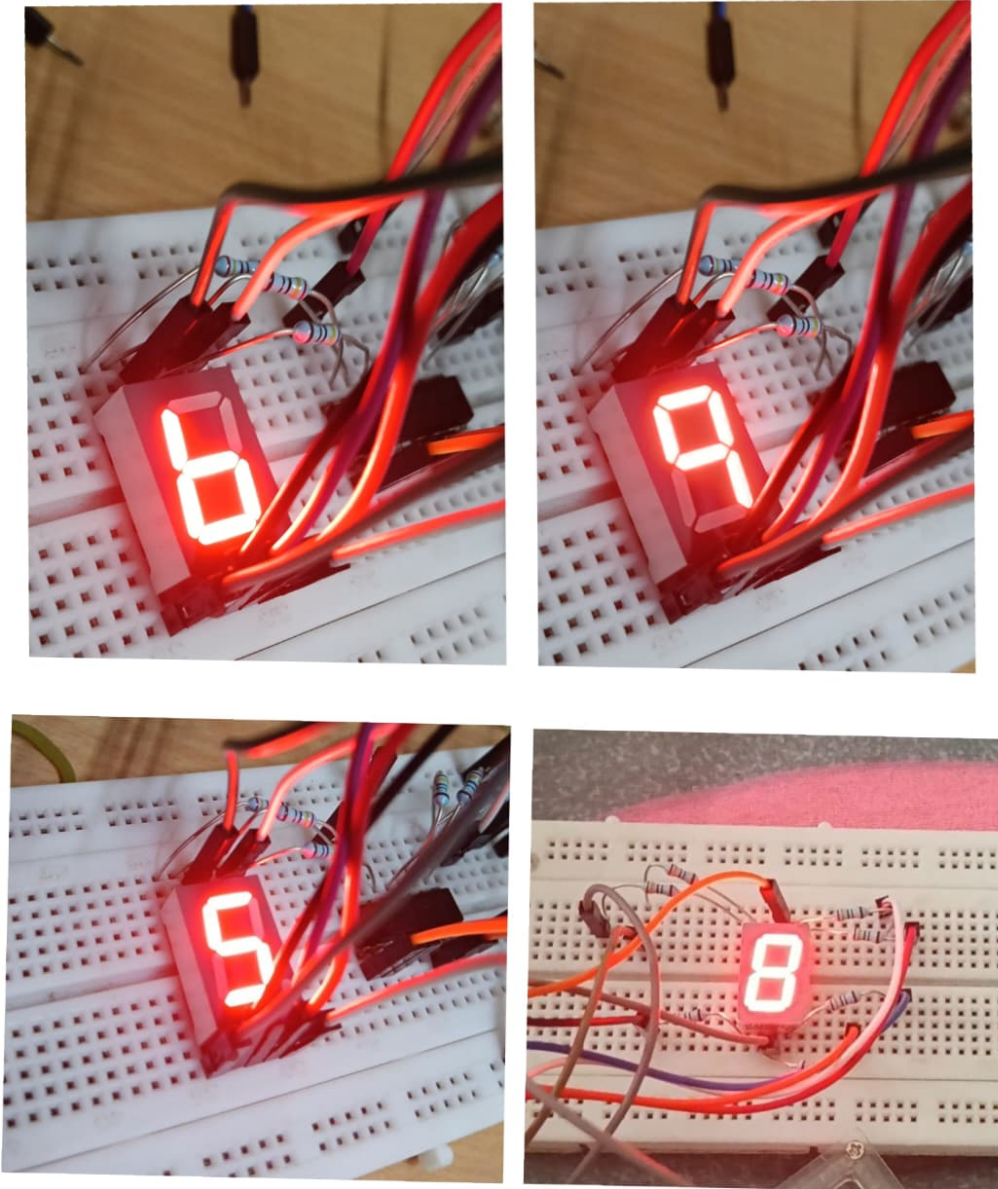


Figure 7: Testing digit 0-9

Step 4: Interfacing the 7447

The 7447 IC was connected between the BCD outputs and the seven-segment display.

The BCD inputs were verified systematically using known values.

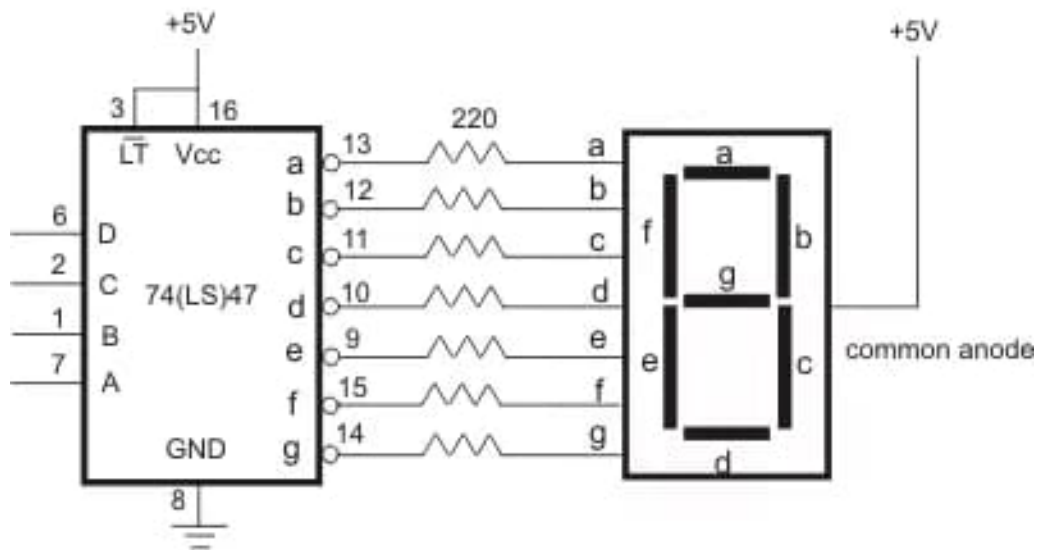


Figure 8: IC 7447

Step 5: Multiple Display Integration

After the individual display and decoder were verified, multiple display positions were introduced.

The display-control arrangement was then tested to determine whether each digit appeared in its correct position.

Step 6: Push-Button Integration

The four push buttons were connected and tested individually.

The buttons were assigned the following functions:

Table 3: Push-button functions

Button	Arduino Pin	Function
Select	D12	Select operating/edit mode
Increment	A2	Increase selected value
Decrement	A0	Decrease selected value
Pause/Resume	A1	Pause or resume clock

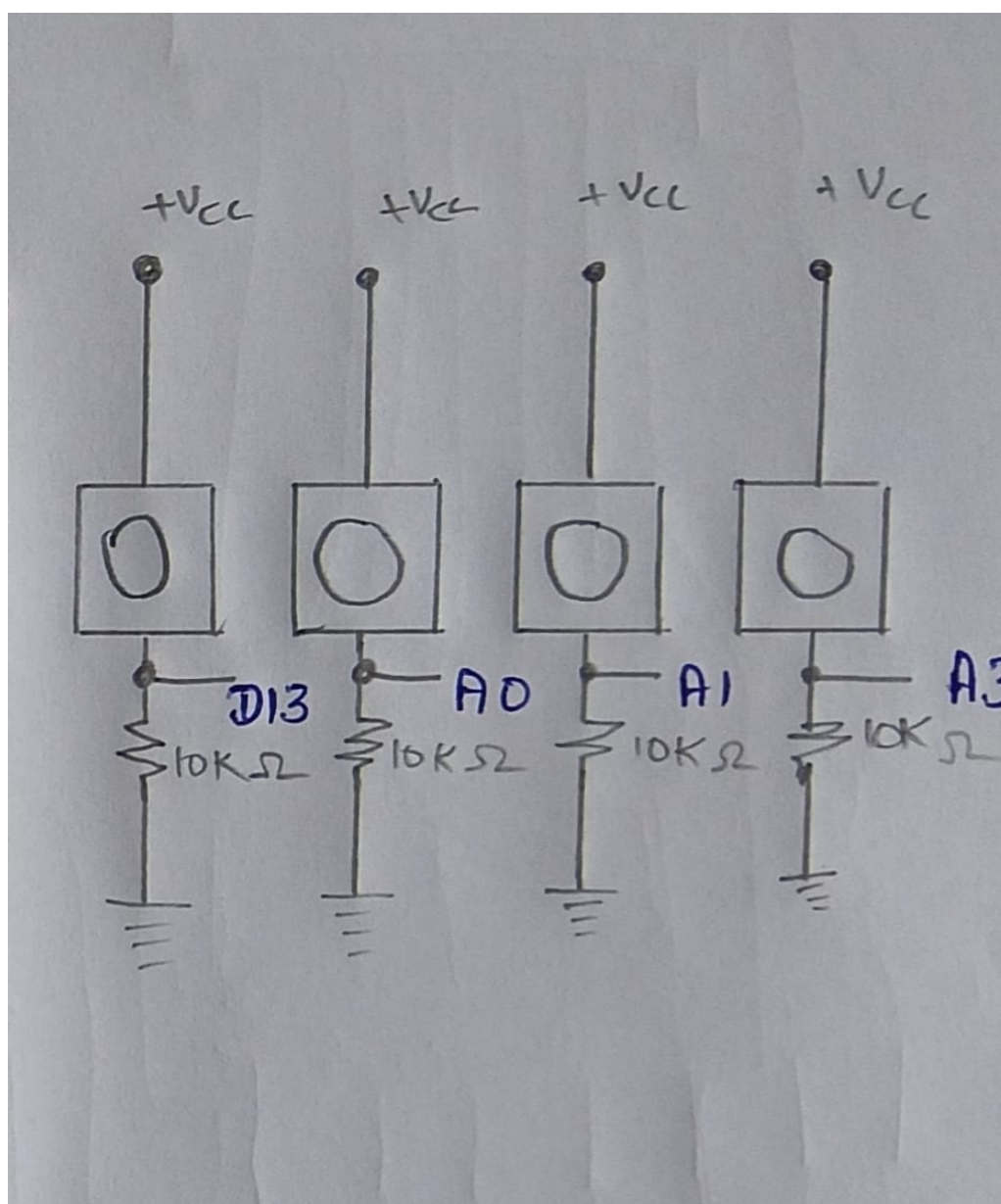


Figure 9: push-button ckt connection

Step 7: Complete System Test

Finally, the Arduino, decoder, six displays, buttons, and power connections were tested together.

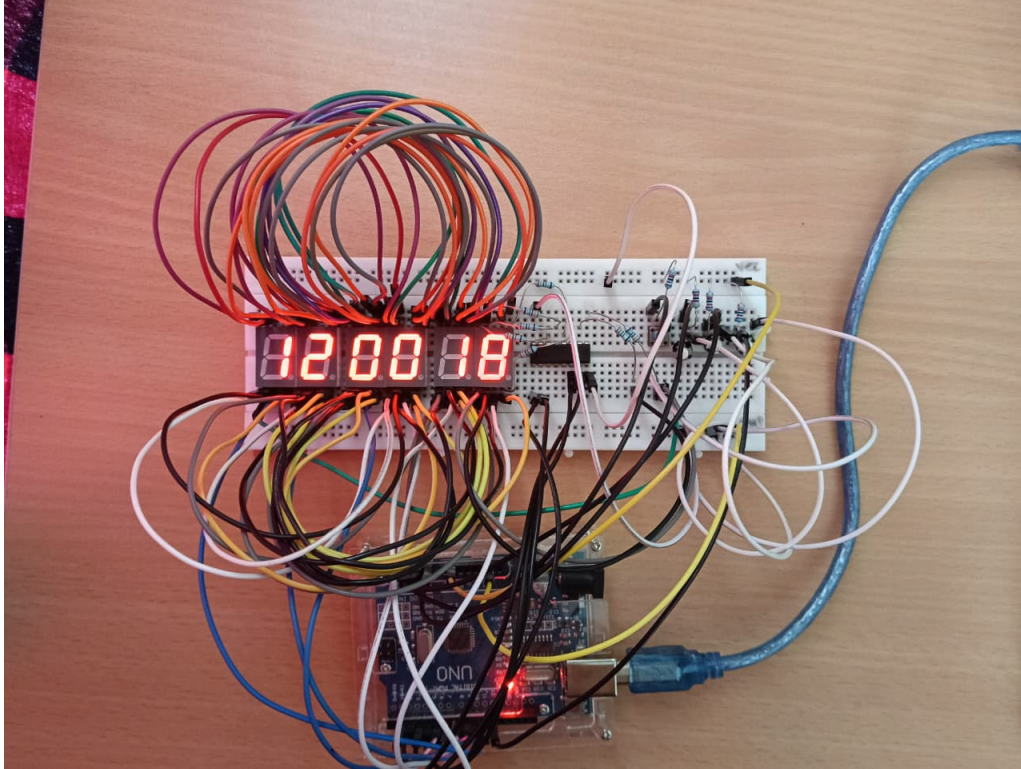


Figure 10: Complete breadboard circuit

5.2 A Note About Careful Changes

One of the major lessons from the project was that hardware changes should be made one section at a time.

Changing several wires simultaneously made it difficult to determine which change caused a new fault.

Therefore, the troubleshooting process followed the general principle:

Observe → Isolate → Test → Correct → Retest

This approach significantly reduced confusion during debugging.

6 Program Logic

The Arduino program is responsible for coordinating the clock state, user inputs, and six-digit display output.

6.1 Core Program Responsibilities

The program performs the following major tasks:

1. Configure the Arduino pins.
2. Configure the four BCD output pins.
3. Configure the six digit-enable pins.
4. Configure the four push-button inputs.
5. Maintain hour, minute, and second variables.
6. Update the time at one-second intervals.
7. Read push-button transitions.
8. Select the required editing mode.
9. Increment or decrement the selected value.
10. Pause or resume the clock.
11. Separate the time into six decimal digits.
12. Convert each digit to BCD.
13. Multiplex the six displays.
14. Blink the selected pair of digits during editing.

6.2 Arduino Pin Configuration

The final program uses the following pin assignments.

Table 4: Arduino pin assignment

Pin	Signal	Function
D2	A	BCD input A (LSB)
D3	B	BCD input B
D4	C	BCD input C
D5	D	BCD input D (MSB)
D6	Digit 6	Seconds ones
D7	Digit 5	Seconds tens
D8	Digit 4	Minutes ones
D9	Digit 3	Minutes tens
D10	Digit 2	Hours ones
D11	Digit 1	Hours tens
D12	Select	Mode selection
A0	Decrement	Decrease selected value
A1	Pause	Pause/resume
A2	Increment	Increase selected value

6.3 Detailed Electrical Connections

The main electrical connections between the Arduino UNO, the 7447 decoder, the six display positions, and the push buttons are summarised below.

Table 5: Detailed Arduino electrical connections

Arduino Pin	Connected To	Purpose
D2	7447 A	BCD bit A (LSB)
D3	7447 B	BCD bit B
D4	7447 C	BCD bit C
D5	7447 D	BCD bit D (MSB)
D6	Display 6	Seconds units digit
D7	Display 5	Seconds tens digit
D8	Display 4	Minutes units digit
D9	Display 3	Minutes tens digit
D10	Display 2	Hours units digit
D11	Display 1	Hours tens digit
D12	Select button	Mode selection
A0	Decrement button	Decrease selected value
A1	Pause button	Pause/resume clock
A2	Increment button	Increase selected value

6.4 Timekeeping Logic

The program uses the Arduino `millis()` function to measure the passage of time.

The time interval is defined as:

$$\text{interval} = 1000 \text{ ms}$$

Therefore, the seconds variable is increased once every approximately one second when the clock is running in normal mode.

The rollover conditions are:

$$60 \text{ seconds} \rightarrow 0 \text{ seconds}$$

$$60 \text{ minutes} \rightarrow 0 \text{ minutes}$$

and:

24 hours $\rightarrow$ 0 hours

Thus, the clock follows the 24-hour format:

00 : 00 : 00 to 23 : 59 : 59

6.5 Button Edge Detection

The program does not simply react whenever a button is HIGH.

Instead, it compares the current button state with the previous state.

A button action occurs when:

Previous State = LOW

and:

Current State = HIGH

This is called rising-edge detection.

It prevents one physical button press from repeatedly increasing or decreasing a value.

A short software delay of approximately 50 ms is also used for basic button debouncing.

6.6 Edit Modes

The variable `editMode` stores the current operating state.

Table 6: Edit mode operation

Mode	State	Function
0	Normal Run	Clock operates normally
1	Edit Hours	Modify hours
2	Edit Minutes	Modify minutes
3	Edit Seconds	Modify seconds

The Select button changes the state according to:

$$\text{editMode} = (\text{editMode} + 1) \bmod 4$$

This creates the cycle:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$$

6.7 Increment and Decrement Logic

The increment button increases only the currently selected quantity.

For hours:

$$H = (H + 1) \bmod 24$$

For minutes:

$$M = (M + 1) \bmod 60$$

For seconds:

$$S = (S + 1) \bmod 60$$

The decrement operation uses wrap-around behaviour.

For example, decrementing hour zero produces:

$$0 \rightarrow 23$$

Similarly:

$$0 \text{ minutes} \rightarrow 59 \text{ minutes}$$

and:

$$0 \text{ seconds} \rightarrow 59 \text{ seconds}$$

6.8 Pause and Resume

The Pause/Resume button changes the Boolean variable `isPaused`.

The two states are:

`isPaused = false`

for normal operation and:

`isPaused = true`

for paused operation.

When the clock is paused, the timekeeping counter does not increment.

6.9 Digit Separation

The six digits are generated from the hour, minute, and second variables.

For hours:

$$H_{10} = \left\lfloor \frac{H}{10} \right\rfloor$$

$$H_1 = H \bmod 10$$

For minutes:

$$M_{10} = \left\lfloor \frac{M}{10} \right\rfloor$$

$$M_1 = M \bmod 10$$

For seconds:

$$S_{10} = \left\lfloor \frac{S}{10} \right\rfloor$$

$$S_1 = S \bmod 10$$

The resulting six-element array is:

$$[D_0, D_1, D_2, D_3, D_4, D_5]$$

corresponding to:

$$[H_{10}, H_1, M_{10}, M_1, S_{10}, S_1]$$

6.10 BCD Generation

The function `writeBCD()` converts a decimal digit into four binary signals.

The four bits are extracted using bitwise operations:

$$A = D_0$$

$$B = D_1$$

$$C = D_2$$

$$D = D_3$$

where the corresponding bit weights are:

$$1, \quad 2, \quad 4, \quad 8$$

These signals are sent to the four BCD input pins of the 7447.

6.11 Display Refresh

The function `refreshDisplay()` updates all six display positions.

For each display:

1. The required digit is selected.
2. The digit is converted into BCD.

3. The corresponding display-enable pin is activated.
4. The display remains active for approximately 2 ms.
5. The display-enable pin is turned off.
6. The next digit is selected.

This process is repeated continuously.

7 Testing, Observations, and Troubleshooting

Testing was performed progressively rather than waiting until the entire circuit was assembled.

7.1 How We Tested

The following testing procedure was used.

Table 7: Functional testing results

Test	Function	Expected Result	Result
T1	Power ON	Display and controller operate normally	Passed
T2	Individual digits	Digits 0–9 are displayed correctly	Passed
T3	BCD decoder	Correct BCD input produces the intended digit	Passed
T4	Seconds counting	Seconds increase once per second	Passed
T5	Minute rollover	00:59:59 changes to 01:00:00	Passed
T6	Hour rollover	23:59:59 changes to 00:00:00	Passed
T7	Increment button	Selected value increases by one per press	Passed
T8	Decrement button	Selected value decreases by one per press	Passed
T9	Pause/resume	Clock stops and resumes correctly	Passed
T10	Edit mode	Selected digits blink during editing	Passed
T11	Six-digit multiplexing	All six positions appear stable	Passed

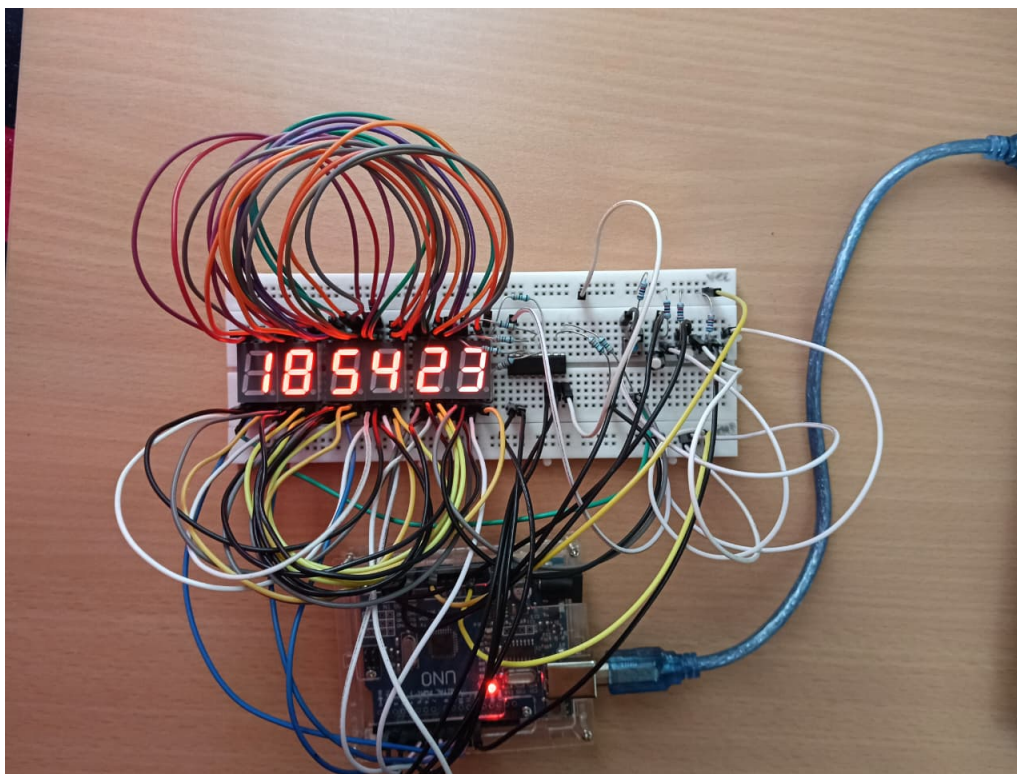


Figure 11: Final testing photograph

7.2 Useful Troubleshooting Ideas

Problem 1: Incorrect Display Type / Common Connection

Initially, identifying the exact display configuration was important. A seven-segment display cannot be interfaced correctly unless its common connection and segment arrangement are known.

Solution:

The display was checked independently and the common connection and segment pins were verified before continuing with the complete circuit.

Problem 2: Incorrect Representation of 6 and 9

One of the most noticeable issues during testing was that the digits 6 and 9 did not initially appear in the desired form.

This indicated that the problem had to be investigated at the segment-mapping and display-control level rather than assuming that the entire circuit was faulty.

Solution:

The segment combinations were individually checked and compared with the expected digit patterns. The relevant connections and logic were then corrected and retested.

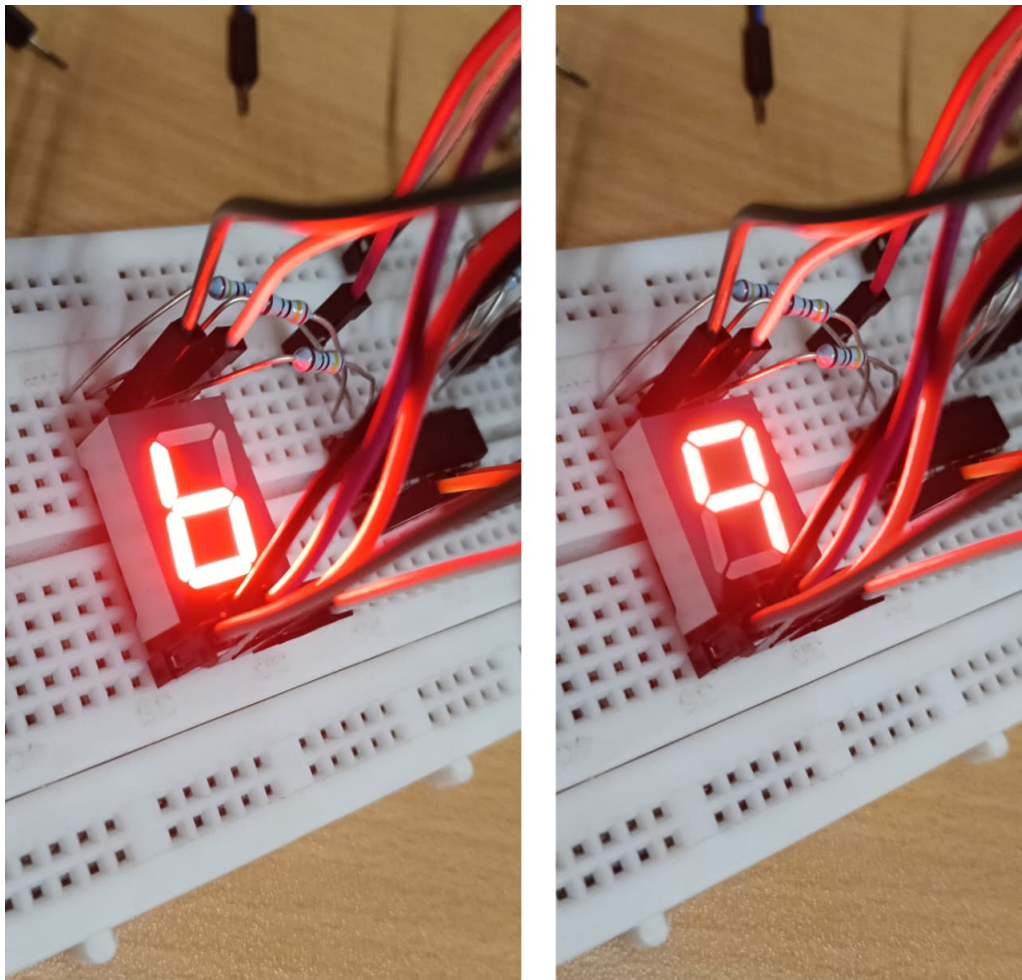


Figure 12: Comparison showing the 6 and 9 display issue

Problem 3: Display Flickering

Flickering was observed during the development of the multi-display system.

Possible causes included:

- Incorrect digit-selection timing.
- Loose breadboard connections.
- Incorrect display-control sequence.
- Unstable or rapidly changing control signals.

Solution:

The display-control sequence was checked systematically, and the breadboard connections were inspected and corrected.

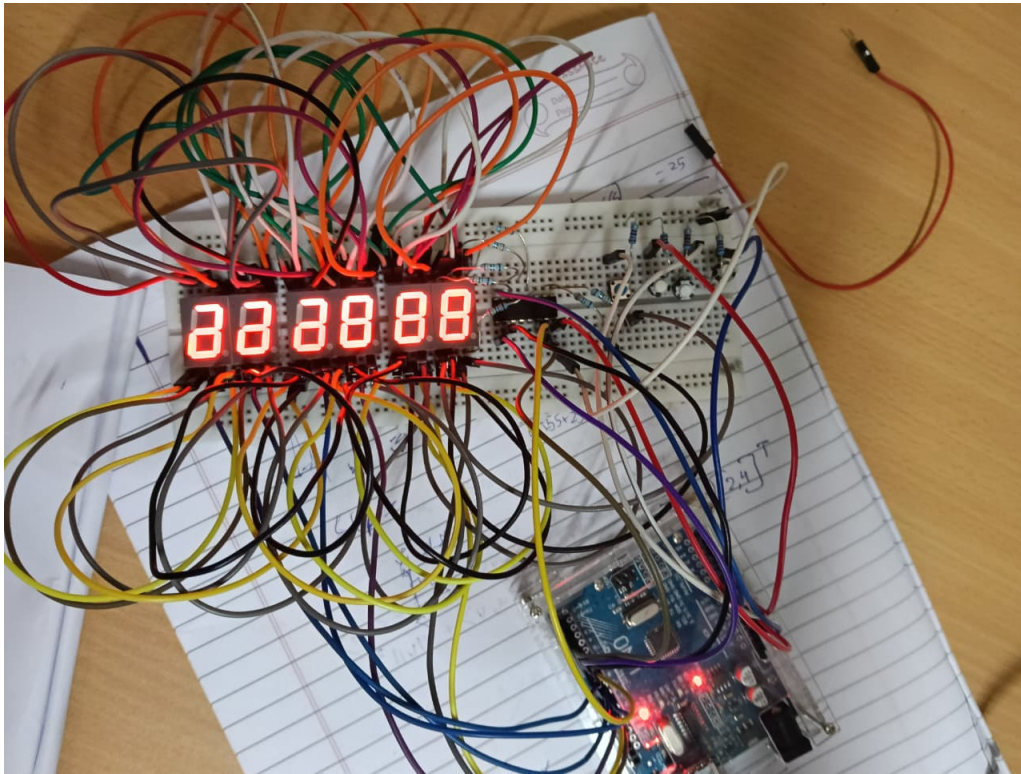


Figure 13: Multi-display wiring during debugging

Problem 4: Incorrect Digit or Segment

At different stages, some displayed values did not correspond to the intended digit.

This can occur because of:

- Incorrect BCD bit order.
- Incorrect Arduino-to-7447 wiring.
- Incorrect segment connection.
- Loose jumper connections.

Solution:

Known BCD values were applied individually and the resulting display was compared with the expected digit. This helped isolate the faulty connection.

Problem 5: Push Button Behaviour

Push-button inputs can produce multiple rapid electrical transitions during a single physical press. This can cause a counter to change more than once.

Solution:

The program uses previous-state tracking to detect only the LOW-to-HIGH transition. A short 50 ms delay was also introduced for basic software debouncing.

Problem 6: Breadboard Wiring Errors

The large number of connections required for the project increased the possibility of wiring mistakes.

Solution:

The circuit was divided into smaller sections:

Arduino → BCD/7447 → Display → Buttons

Each section was verified before proceeding to the next stage.

8 Results

The completed prototype successfully displayed time in the HH:MM:SS format using six seven-segment displays.

The normal clock operation was tested over the 24-hour range from 00:00:00 to 23:59:59. The seconds, minutes, and hours counters were verified during normal operation and at their rollover conditions.

The four push buttons successfully provided the required user functions: mode selection, increment, decrement, and pause/resume. The selected pair of digits also provided a blinking indication during editing.

The six display positions were controlled using multiplexing. Each digit was activated sequentially for approximately 2 ms, producing an approximate complete-display refresh frequency of 83.3 Hz.

The final system therefore demonstrated successful integration of the Arduino UNO, BCD decoder, six seven-segment displays, push buttons, timing logic, and multiplexed display control.

9 What We Learned as a Team

9.1 Technical Learning

The project provided practical experience in several areas of electrical and electronics engineering.

- Seven-segment display interfacing.
- Binary Coded Decimal representation.
- BCD-to-seven-segment decoding.
- Arduino UNO GPIO control.
- Digital timing concepts.
- Multiplexing and display scanning.
- Push-button interfacing.
- Current limiting for LED displays.
- Breadboard circuit construction.
- Hardware debugging.

The project also demonstrated the importance of understanding the relationship between hardware and software.

A correct program cannot compensate for incorrect wiring, and a correctly wired circuit cannot operate correctly if the control logic is wrong.

9.2 Working Together

The project was developed as a team effort.

The work involved:

- Understanding the circuit.
- Checking component connections.
- Testing individual sections.

- Identifying faults.
- Discussing possible causes.
- Implementing corrections.
- Retesting the complete system.

The debugging process highlighted the importance of communication and systematic testing when working on a hardware project.

10 Our Project Development Diary

10.1 Small Steps Made the Final Result Easier

The project was not completed successfully in a single attempt. Instead, it evolved through multiple stages.

The development can be summarised as:

**Component Identification → Single Display → Digit Testing → 7447
Interface → Multiple Displays → Buttons → Clock → Debugging → Final
Testing**

Each stage provided information that was useful for the following stage.

Testing the individual display before constructing the complete system was especially useful because it reduced the number of possible causes when a problem appeared later.

10.2 How We Kept Each Other Moving

During troubleshooting, some problems required repeated testing.

Instead of changing the entire circuit, the system was divided into smaller sections and each section was checked separately.

This created a structured debugging method:

Identify Problem → Form Hypothesis → Test → Fix → Verify
--

This approach helped transform confusing hardware behaviour into specific, testable problems.

11 Conclusion

The Arduino UNO-based digital clock demonstrates the integration of a microcontroller with seven-segment displays, BCD decoding, user inputs, and display multiplexing.

The project provided practical experience beyond theoretical understanding. The construction process required careful attention to component identification, wiring, signal mapping, timing, display behaviour, and systematic debugging.

The final program successfully provides normal clock operation along with user-controlled editing of hours, minutes, and seconds. The pause/resume function provides additional control, while edge detection prevents unwanted multiple increments from a single button press.

One of the most valuable parts of the project was troubleshooting. Problems such as incorrect digit representation, display flickering, wiring errors, and button behaviour required the circuit to be analysed step by step.

The project therefore achieved not only the objective of constructing a digital clock but also the broader objective of understanding how digital hardware systems are designed, assembled, tested, and debugged.

11.1 Final Project Snapshot

Table 8: Final project summary

Parameter	Description
Controller	Arduino UNO
Display	Six seven-segment digits
Time format	HH:MM:SS
Decoder	7447 BCD-to-seven-segment decoder
BCD pins	D2–D5
Digit enable pins	D6–D11
User input	Four push buttons
Clock range	00:00:00–23:59:59
Construction	Breadboard-based
Timing interval	1000 ms
Display refresh	Approximately 2 ms per digit
Programming	Arduino IDE / Arduino C++

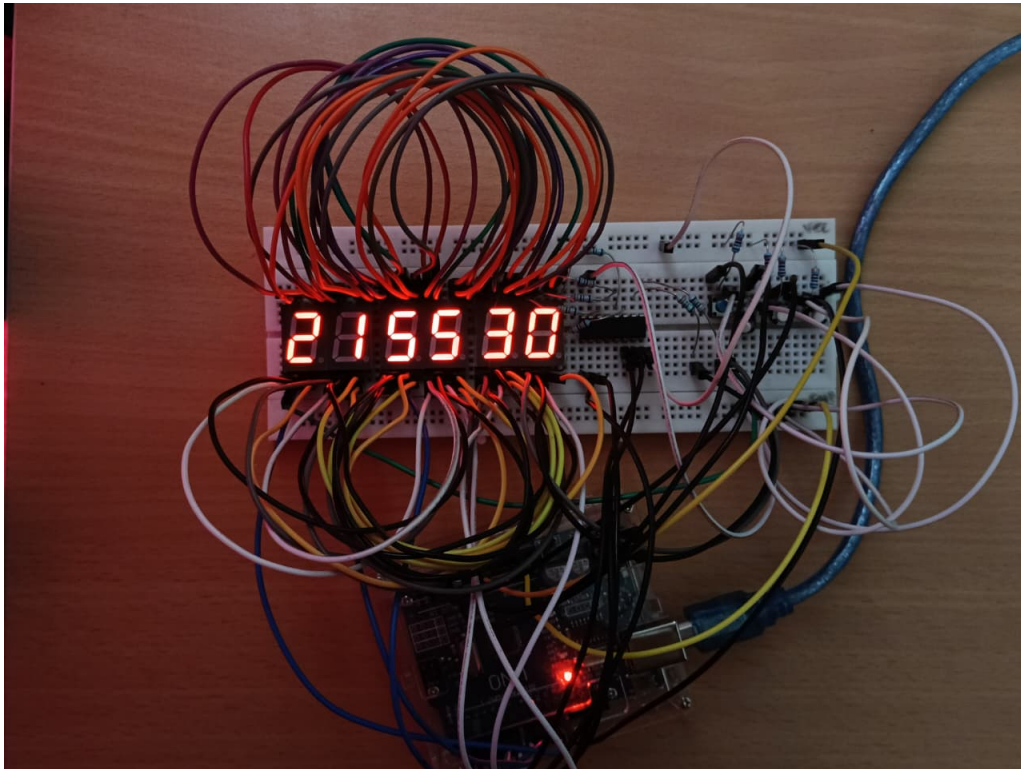


Figure 14: Final working project

11.2 Present Limitations

The present prototype has several limitations:

- The circuit is constructed on a breadboard.
- The clock does not use a dedicated real-time clock module.
- Long-term timing accuracy has not been characterised against a precision reference.
- The prototype does not use a professionally manufactured PCB.
- The current implementation is primarily a laboratory prototype.
- The current timekeeping implementation is based on the Arduino `millis()` function rather than a dedicated real-time clock (RTC) module, so long-term timing accuracy may experience some drift.

11.3 Future Improvements

Several improvements could be incorporated into a future version.

1. **Real-Time Clock Module:** A dedicated RTC module could improve long-term timekeeping accuracy and allow the clock to retain the time when power is removed.
2. **PCB Implementation:** The breadboard prototype could be converted into a compact PCB to reduce wiring and improve reliability.
3. **Alarm Function:** An alarm feature could be added using a buzzer.
4. **Brightness Control:** Automatic or manually controlled display brightness could be implemented.
5. **Colon Indicator:** A dedicated colon indicator could improve readability of the HH:MM:SS format.
6. **Long-Term Accuracy Testing:** The clock could be compared with a reference clock over several days to measure timing drift.
7. **Improved Enclosure:** A professionally designed enclosure could be added in a future version to improve the physical appearance and protection of the electronics.

References

1. Arduino Documentation, <https://docs.arduino.cc/>
2. 7447 BCD-to-Seven-Segment Decoder/Driver Datasheet.
3. Project laboratory and practical material provided for the digital electronics and Arduino project.
4. Indian Institute of Technology Hyderabad, Department of Electrical Engineering, course and laboratory material.

A Final Circuit Diagram

The final circuit diagram of the project is provided below.

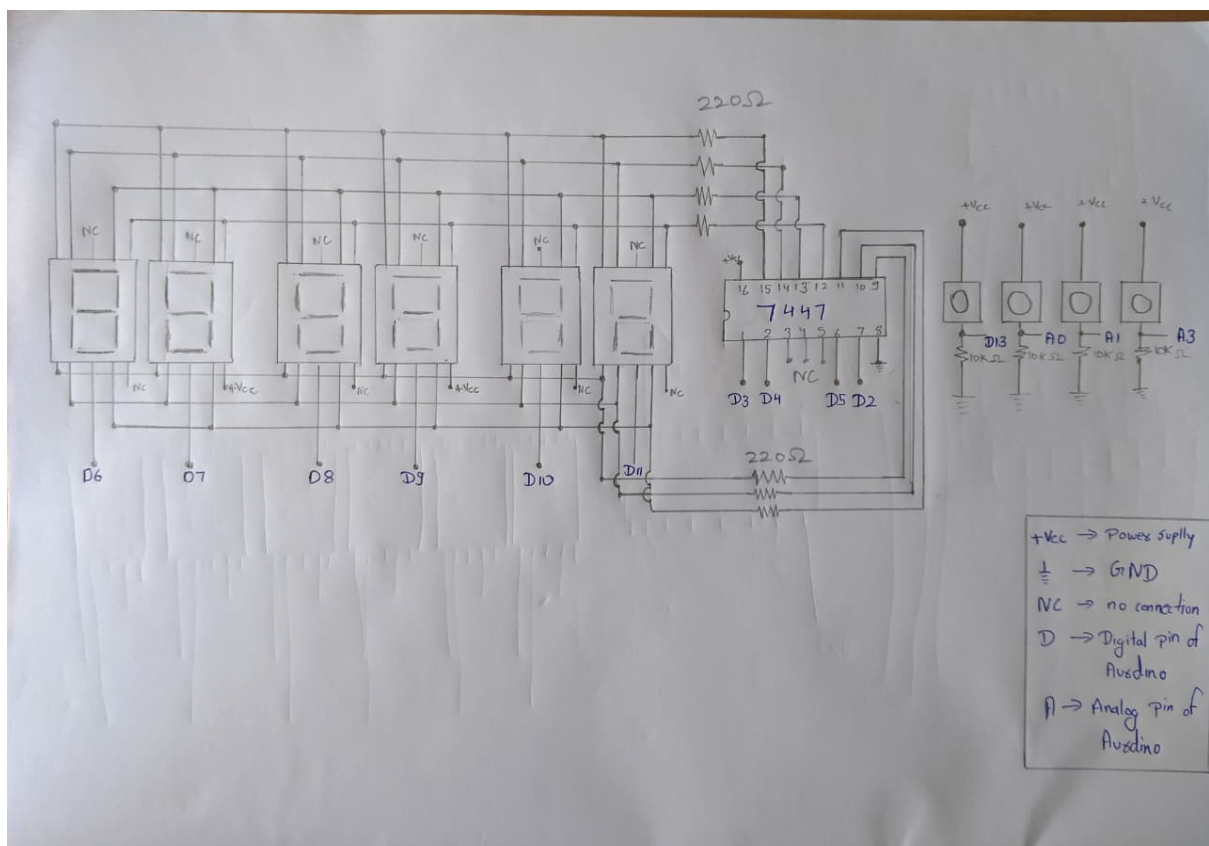


Figure 15: Final circuit diagram of the Arduino UNO-based digital clock

B Final Hardware Photographs

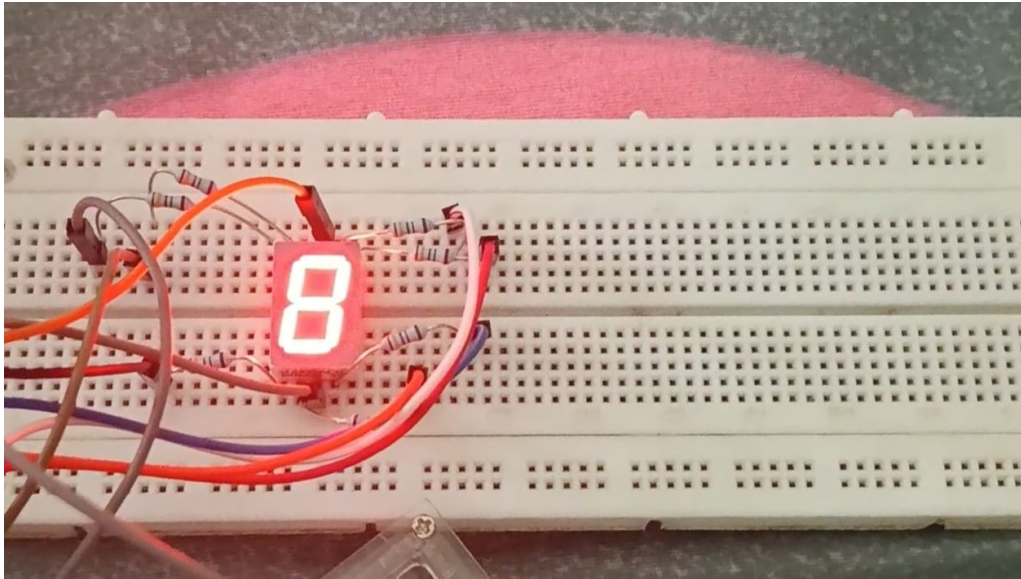


Figure 16: Breadboard during initial construction

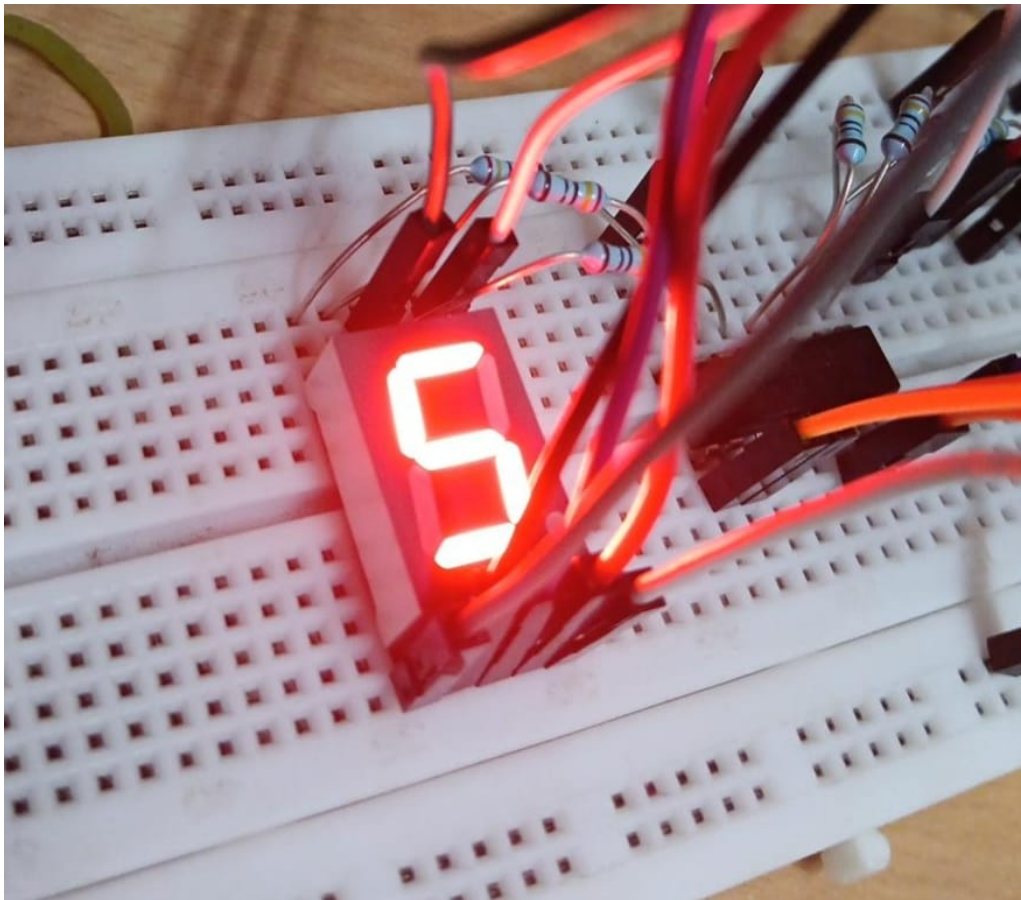


Figure 17: Display testing

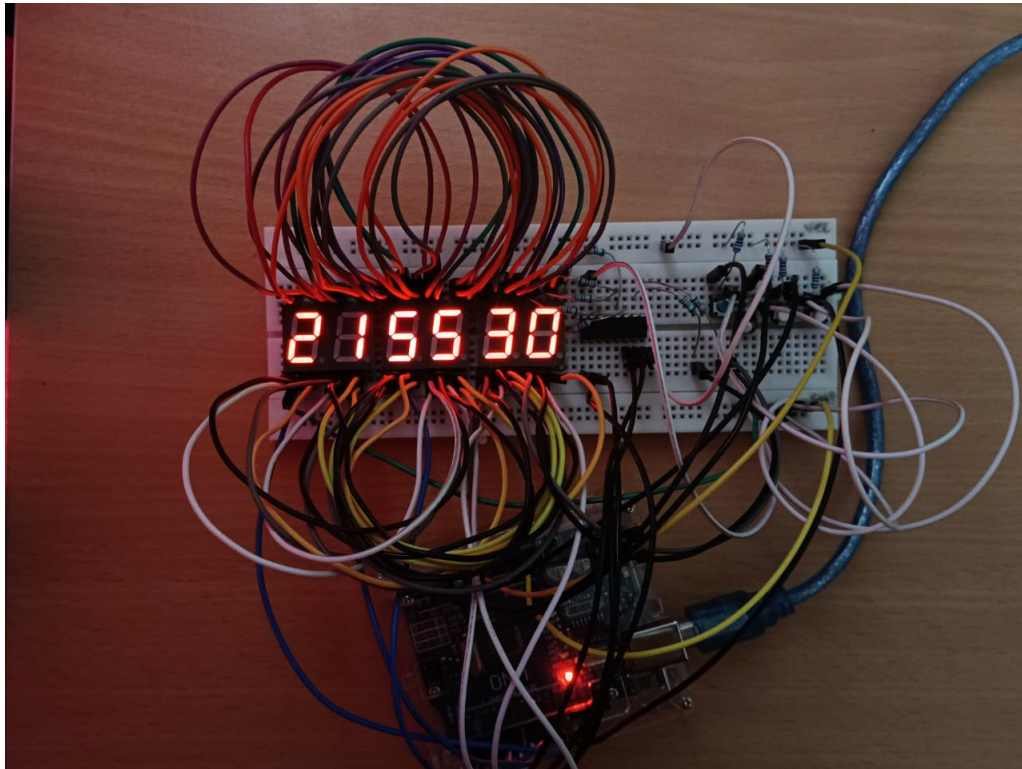


Figure 18: final project image

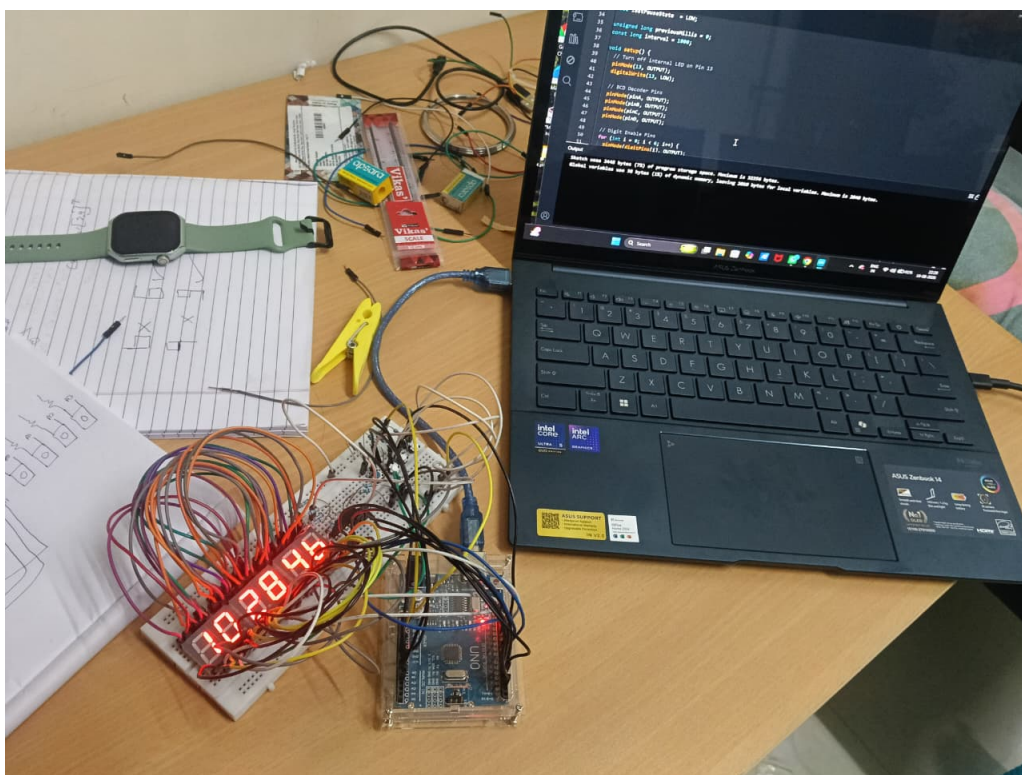


Figure 19: Final working digital clock prototype

C Complete Arduino Source Code

The complete Arduino program used in the final prototype is provided below.

```

1
2 // =====
3 // Fixed 6-Digit Clock Code (HH:MM:SS Format)
4 // =====
5
6 // 1. Decoder Pin Definitions (BCD)
7 const int pinA = 2; // Input A (LSB)
8 const int pinB = 3; // Input B
9 const int pinC = 4; // Input C
10 const int pinD = 5; // Input D (MSB)
11
12 // 2. Digit Enable Pins (Left to Right: HH MM SS)
13 const int digitPins[6] = {11, 10, 9, 8, 7, 6};
14
15 // 3. Pushbutton Pins (Active HIGH)
16 const int btnSelect = 12; // Button 1: Mode/Select
17 const int btnInc     = A2; // Button 2: Increment (+)
18 const int btnDec     = A0; // Button 3: Decrement (-)
19 const int btnPause  = A1; // Button 4: Pause / Resume
20
21 // 4. Timekeeping Variables
22 int hours    = 12; // Start at 12
23 int minutes  = 0;
24 int seconds  = 0;
25
26 // 5. System States
27 int editMode = 0; // 0 = Normal Run, 1 = Edit Hours, 2 = Edit Minutes
28 //      , 3 = Edit Seconds
29 bool isPaused = false;
30
31 // Button state tracking for edge detection (Prevents +10 jumps)
32 bool lastSelectState = LOW;
33 bool lastIncState    = LOW;
34 bool lastDecState    = LOW;
35 bool lastPauseState  = LOW;
36
37 unsigned long previousMillis = 0;
38 const long interval = 1000;
39
40 void setup() {
41     // Turn off internal LED on Pin 13
42     pinMode(13, OUTPUT);
43     digitalWrite(13, LOW);

```

```

43
44 // BCD Decoder Pins
45 pinMode(pinA, OUTPUT);
46 pinMode(pinB, OUTPUT);
47 pinMode(pinC, OUTPUT);
48 pinMode(pinD, OUTPUT);
49
50 // Digit Enable Pins
51 for (int i = 0; i < 6; i++) {
52     pinMode(digitPins[i], OUTPUT);
53     digitalWrite(digitPins[i], LOW);
54 }
55
56 // Button Inputs
57 pinMode(btnSelect, INPUT);
58 pinMode(btnInc, INPUT);
59 pinMode(btnDec, INPUT);
60 pinMode(btnPause, INPUT);
61 }
62
63 void loop() {
64     checkButtons();
65
66     // 1-Second Timekeeping Counter Logic
67     if (!isPaused && editMode == 0) {
68         unsigned long currentMillis = millis();
69         if (currentMillis - previousMillis >= interval) {
70             previousMillis = currentMillis;
71
72             seconds++;
73             if (seconds >= 60) {
74                 seconds = 0;
75                 minutes++;
76                 if (minutes >= 60) {
77                     minutes = 0;
78                     hours++;
79                     if (hours >= 24) { // Strictly caps hours at 24 (0-23)
80                         hours = 0;
81                     }
82                 }
83             }
84         }
85     }
86
87     refreshDisplay();
88 }
89
    
```

```

90 void writeBCD(int val) {
91     digitalWrite(pinA, (val & 0x01) ? HIGH : LOW);
92     digitalWrite(pinB, (val & 0x02) ? HIGH : LOW);
93     digitalWrite(pinC, (val & 0x04) ? HIGH : LOW);
94     digitalWrite(pinD, (val & 0x08) ? HIGH : LOW);
95 }
96
97 // Fixed Button Handling Logic (Triggers ONLY once per press)
98 void checkButtons() {
99     bool currentSelect = digitalRead(btnSelect);
100     bool currentInc     = digitalRead(btnInc);
101     bool currentDec     = digitalRead(btnDec);
102     bool currentPause   = digitalRead(btnPause);
103
104     // Button 1: Select / Mode (Triggers on press transition LOW -> HIGH)
105     if (currentSelect == HIGH && lastSelectState == LOW) {
106         editMode = (editMode + 1) % 4;
107         delay(50); // Small debounce
108     }
109
110     // Button 2: Increment (+1)
111     if (currentInc == HIGH && lastIncState == LOW) {
112         if (editMode == 1) hours = (hours + 1) % 24; // 0 to 23
113         else if (editMode == 2) minutes = (minutes + 1) % 60; // 0 to 59
114         else if (editMode == 3) seconds = (seconds + 1) % 60; // 0 to 59
115         delay(50);
116     }
117
118     // Button 3: Decrement (-1)
119     if (currentDec == HIGH && lastDecState == LOW) {
120         if (editMode == 1) hours = (hours == 0) ? 23 : hours - 1;
121         else if (editMode == 2) minutes = (minutes == 0) ? 59 : minutes -
122             1;
123         else if (editMode == 3) seconds = (seconds == 0) ? 59 : seconds -
124             1;
125         delay(50);
126     }
127
128     // Button 4: Pause / Resume
129     if (currentPause == HIGH && lastPauseState == LOW) {
130         isPaused = !isPaused;
131         delay(50);
132     }
133
134     // Store current states for next check
135     lastSelectState = currentSelect;
136     lastIncState    = currentInc;

```

```

135     lastDecState      = currentDec;
136     lastPauseState   = currentPause;
137 }
138
139 // Display Multiplexing HH MM SS
140 void refreshDisplay() {
141     int digits[6];
142
143     // Digit Assignment strictly HH MM SS (Left to Right)
144     digits[0] = hours / 10;    // Hour Tens
145     digits[1] = hours % 10;    // Hour Ones
146     digits[2] = minutes / 10; // Min Tens
147     digits[3] = minutes % 10; // Min Ones
148     digits[4] = seconds / 10; // Sec Tens
149     digits[5] = seconds % 10; // Sec Ones
150
151     for (int i = 0; i < 6; i++) {
152         // Visual Blinking indicator during Edit Mode
153         if (editMode == 1 && (i == 0 || i == 1) && (millis() % 400 < 200))
154             continue;
155         if (editMode == 2 && (i == 2 || i == 3) && (millis() % 400 < 200))
156             continue;
157         if (editMode == 3 && (i == 4 || i == 5) && (millis() % 400 < 200))
158             continue;
159
160         writeBCD(digits[i]);
161         digitalWrite(digitPins[i], HIGH);
162         delayMicroseconds(2000);
163         digitalWrite(digitPins[i], LOW);
164     }
165 }

```

Listing 1: Complete Arduino UNO digital clock source code